

CS3217: Software Engineering on Modern Application Platforms

AY 2023/2024, Semester 2

Sprint Report 2

TowerForge

Name	Matric Number
Keith Gan	[redacted]
Phua Zheng Ze	[redacted]
Rubesh Suresh	[redacted]
Vanessa Mae	[redacted]

Table of Contents

1 – Project Overview **3**
2 – Features and Specifications **4**
3 – User Manual **5**
4 – TowerForge Design Overview **9**
Top Level Design 9
Entity Component System 10
Events API 11
Rendering API 13
Storage API 15
Usage of Libraries 16
Designs Problems 16
Network API 16
Storage API 16
5 – Runtime Structure **17**
6 – Module Structure **19**
7 – Reflection **24**
Lessons 24
Evaluation 24
Overall Reflection 25
Known Bugs and Unresolved Issues 26
8 – Appendix **27**
A – GUI 27
B – Test Cases 28
Unit Tests 29
Integration Testing 30
C – Project Schedule 31
Sprint Objectives 31
D – Individual roles and contribution 32

CS3217 Sprint Report II: TowerForge



1 – Project Overview

Preface

TowerForge is a strategic tower defense game set in a fictional world where players must defend their fortress from waves of enemy invaders. Players strategically place various towers along the enemy's path to destroy them before they breach the fortress gates and lose. As the game progresses, players earn game points to place more fortresses or summon more units and unlock to fend off increasingly challenging waves of enemies.

Overview

We plan to develop a grid based tower defense builder. Players should be able to create their own levels with a level builder as well as specify the rules that govern the game world such as positions where towers can be placed and the like. We hope to allow players to be able to make relatively simple customisations within the application and provide an easy to extend source code for players to further customize the game to their vision.

Basic Application Usage

In the level designer, players can specify parameters such as path arrangements, enemy types, enemy counts, minion timer and victory conditions, etc. Players will be given the ability to design the specific rules according to which the game will be played. This way, the players will have maximum control over the exact kind of game experience that they want to have.

Gameplay

During the game itself, players can place units (minions) to **attack**. Players can decide where they want minions in order to attack the opponent player and hence reduce their Health Points (HP). The player whose tower HP reaches zero first will lose the game.

Scope of Sprint 2

Our goal for Sprint 2 was …

2 – Features and Specifications

MeleeUnit	- A movable unit that deals damage on contact with units of opposing team - Will move towards the enemy base. Stops when there is an enemy entity in contact. - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
SoldierUnit	- A movable unit that is able to spawn Bullets to deal damage to entities of the opposing team. - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
WizardUnit	- A movable unit that is able to spawn Magic Balls to deal damage to entities of the opposing team. - Moves much faster than other units - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
ArrowTower	- An immovable unit that can be placed on the map to protect a lane. - Able to spawn Bullets to to deal damage to entities of the opposing team - Takes damage from damage causing entities from opposing team - Disappears when their health reaches 0
Bullet	- A movable projectile that is able to deal damage with entities of the opposing team. - Disappears when dealing damage to an entity of the opposing team
Team	- Represents the player. - Contains the number of lives left by the player. This represents the number of entities that needs to be let into the base for the player to lose - Contains and generates the points that are used to purchase units.
AiComponent	- Acts as the player 2 in single player mode. - Randomly chooses a random unit and spawns it on a random lane on the board provided that it has enough points to spawn the unit. - A random delay is added before it attempts to spawn another unit.

3 – User Manual

Welcome to TowerForge!

Objective

TowerForge is a tower defense game. Your objective is to build an army that can defeat your opponents before the time runs out. In this game you have three base units: Melee, Soldier, and Arrow Tower.

Combat rules

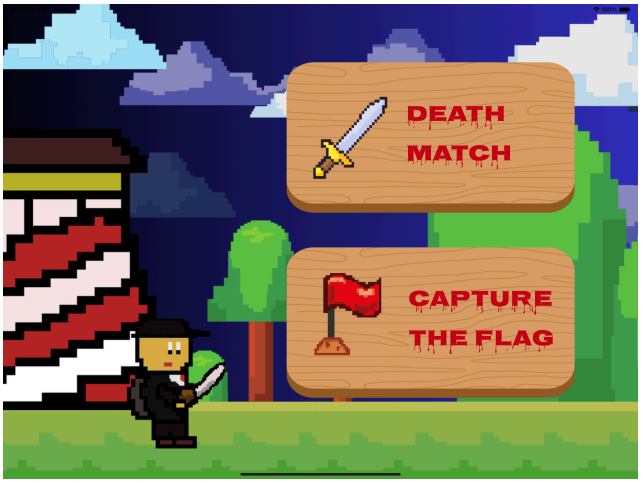
A Melee unit will only begin attacking when it contacts an enemy unit. A Soldier unit will begin firing as soon as an enemy unit is within its firing range. Lastly, an arrow tower is a static defense unit which is able to block enemy units while firing projectiles at incoming enemies.

Game Modes

TowerForge offers diverse gameplay modes with different game mechanics and objectives. Currently, it supports the “DeathMatch” mode and the “Capture the Flag” mode that users can choose at the beginning in the game mode selection screen as shown below.

<u>Capture the Flag</u>

Objective: Capture the opponent’s base while defending their own base. Each player will have their own life points for their base and attack the opponent by placing units so that it reaches the opponent’s base. Upon successful infiltration of the unit into the opponent’s base, the life point of the opponent’s base will be reduced.



Winning condition: The player with zero life point left will lose and the opposite team will win

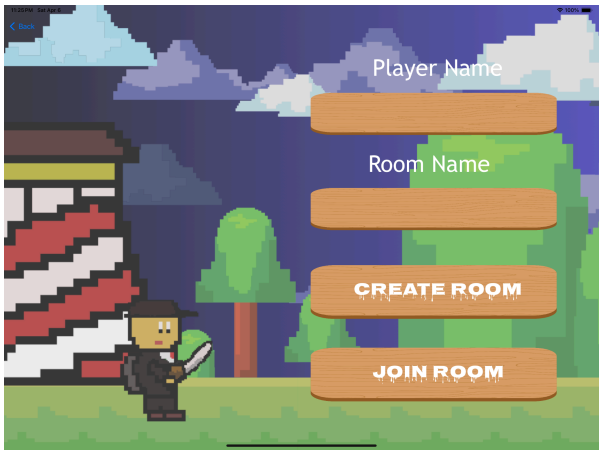
<u>Death Match</u>

Objective: Eliminate as many opponent’s units within the time limit. Eliminating a unit will earn the player points and hence this would contribute to their winning points.

Winning condition: The player with the most point at the end of the game will win

Multiplayer

Players can choose to play with their friends using the multiplayer mode. They will have the option to choose between creating a room or joining an existing room. They can join another room by knowing the name of the room that the other player gives. Each room can only have a maximum of 2 players. They can then start the game to play with other real players. While the default single player will be against the AI system.

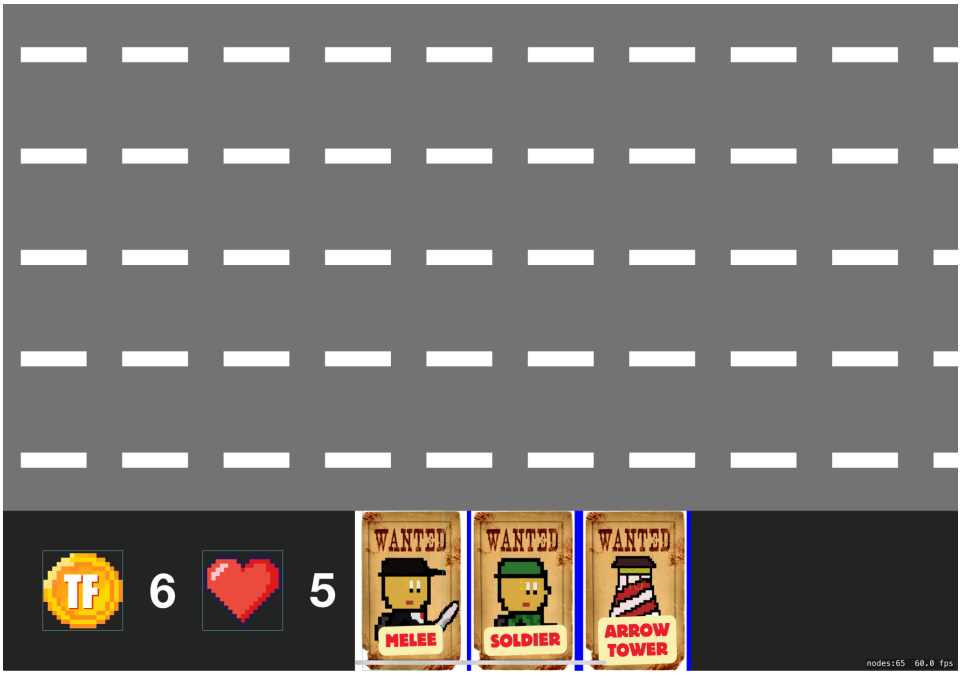

Back
Player Name
Room Name
CREATE ROOM
JOIN ROOM


Back
MIL
Start
LEAVE

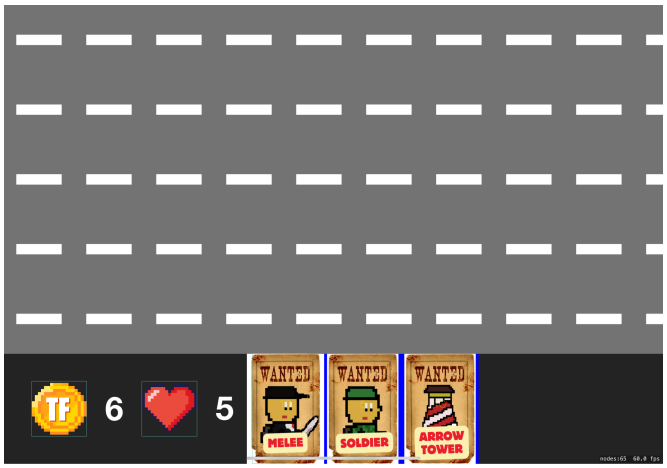
Demo
Shown below is the starting screen which will be shown on app launch.

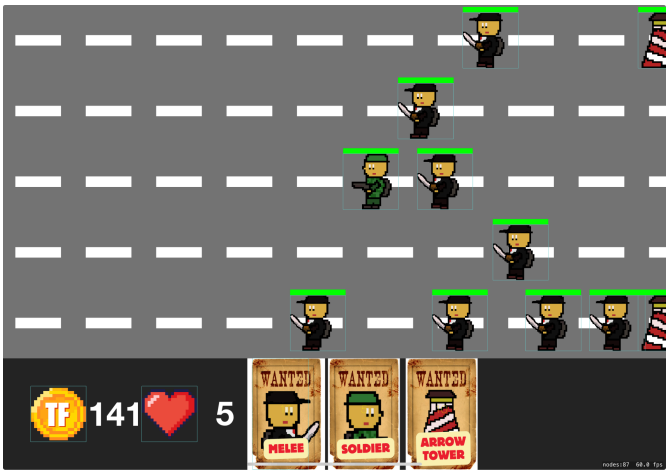


Clicking the play game button will bring you to the game scene.



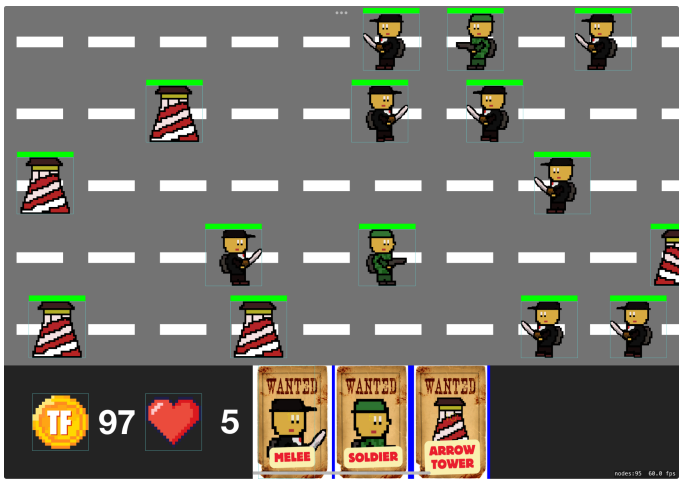
As time goes by, the number of coins you have will increase. You can use these coins to purchase units. Additionally, as time goes by, enemy units will spawn on the right side of the screen and move towards the left side of the screen.

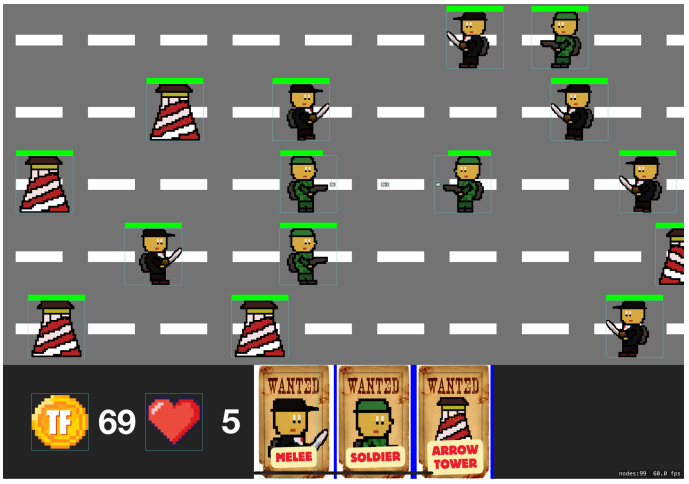




The left side of the screen indicates the user's "base" and the right side of the screen represents the enemy's "base". Enemy units will move towards the user's base and vice versa.

With coins, the user can generate more units and place them on the grid.





As shown on the right, as units come within firing range of each other, they will fire at each other, causing their health points to go down according to the damage dealt. After a certain number of hits, the units will "die" which will be represented by their disappearing from the game board.

Units that survive will continue to march towards their respective opponent bases.

When a unit passes the left edge of the screen, the user's life points will decrease. This is denoted by the number beside the heart in the selection bar at the bottom. When this life point reaches 0, the user will have lost the game.

The same applies to the enemy. Thus, the goal of the game is to make the opponent's life points reach 0. The first one to make the opponent's life points reach 0 wins the game.*

4 – TowerForge Design Overview

Top Level Design

TowerForge consists of 3 main APIs. These include the Entity Component System(ECS), the Events API and the Rendering API. They are shown in Figure 1 below.

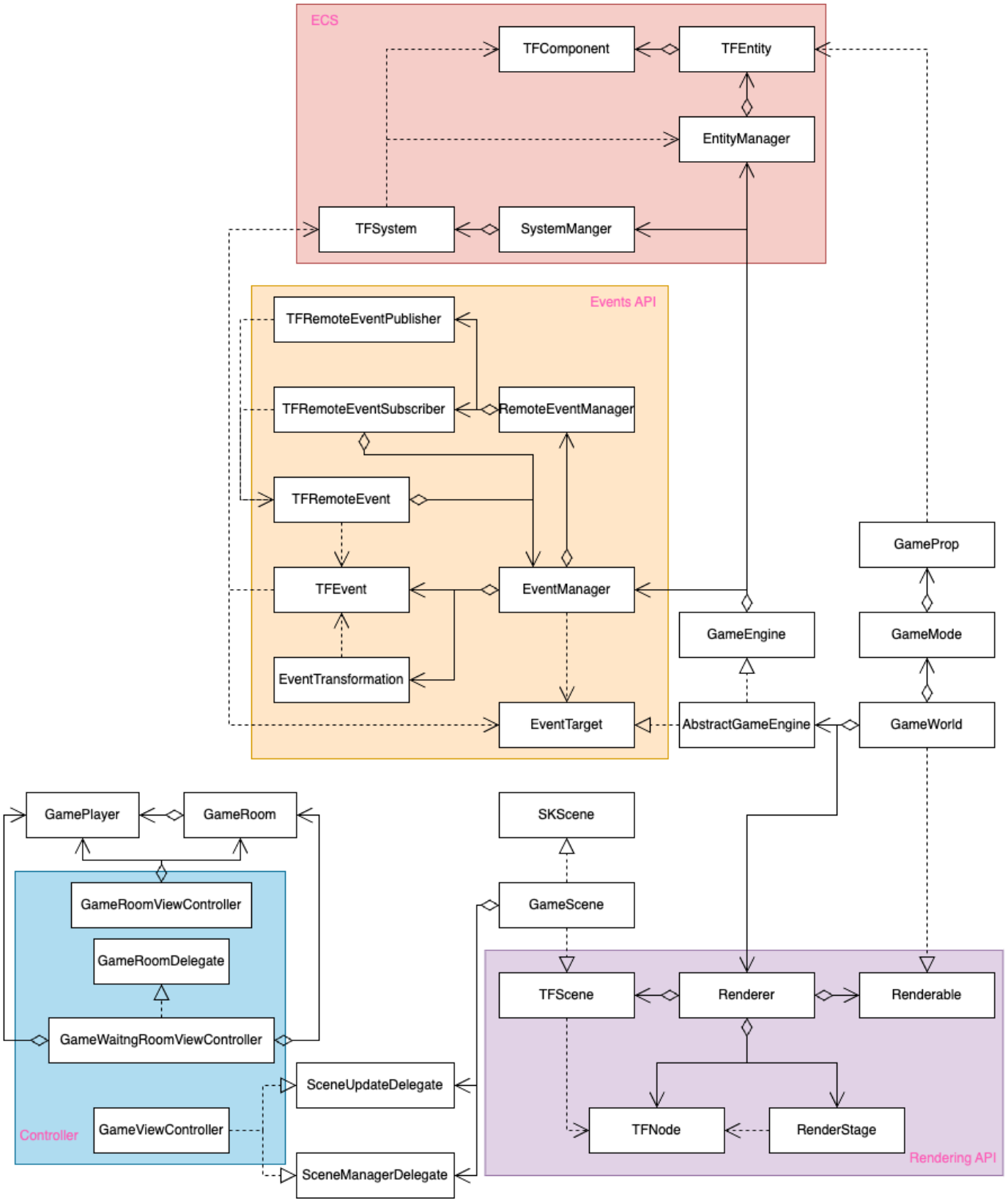


Figure 1: Top Level Class Diagram

Entity Component System

TowerForge is developed upon a robust and well-known game architectural foundation that leverages the **Entity-Component-System (ECS)** paradigm to manage the game entities, behaviors and interactions. **TFEntity** represents the game entities and the fundamental building blocks of the game world / level. **TFComponent** encapsulates the data and behaviors while **TFSystem** orchestrates the processing of entities with specific component compositions so they are more like the features of the game object. This **ECS** framework promotes code reusability and enables simpler addition and removal of features and content.

The **ECS** framework is supported by the **EntityManager** (Figure 2) which holds all the entities that exist in the GameWorld at any point in time. Any **TFEntity** must be added to the **EntityManager** for it to be able to interact with other entities and for it to be able to be rendered. We can then access and remove the entity using its unique identifier. Each entity will also contain a list of **TFComponent**s which defines the behavior of the **TFEntity** that it is attached to. All **TFComponent** have a weak association with their owner **TFEntity** so that we can easily query the **EntityManager** to get the entity that has that specific component.

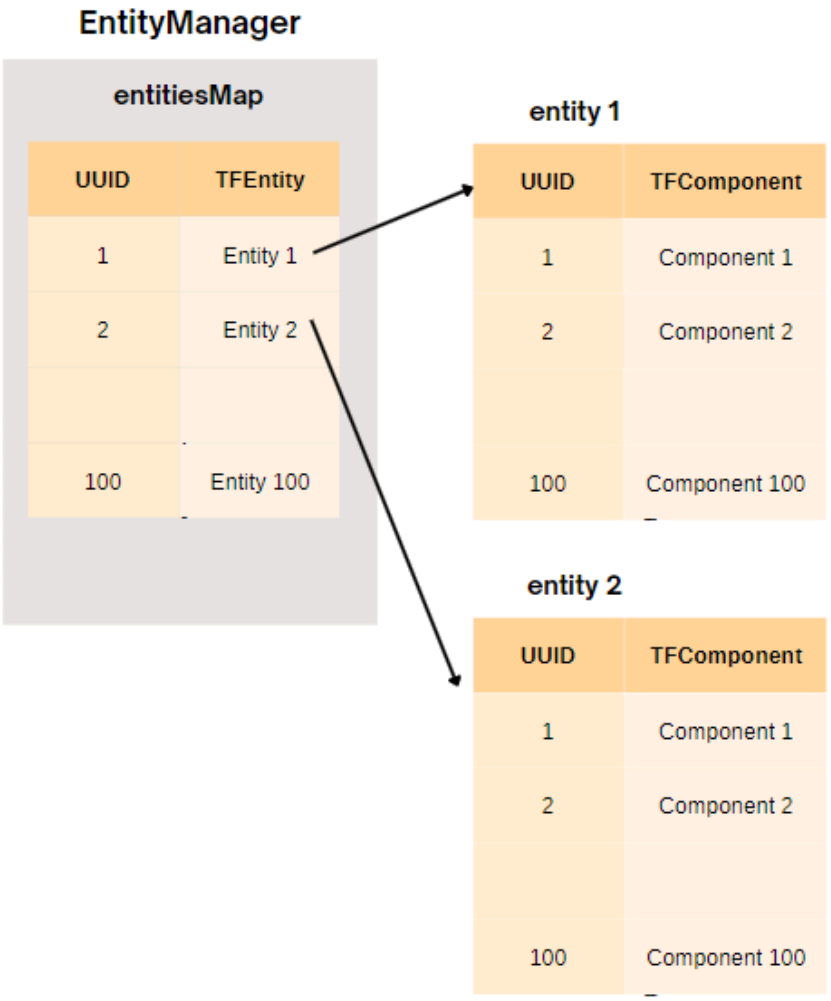


Figure 2: Overview of EntityManager

Each **TFEntity** and **TFComponent** are only to be modified by a **TFSystem**. This is to maintain the integrity of the data store in **TFEntity** and **TFComponent** which represents the game entity in the game world.

Events API

Events API serves as the means of communication between the different **TFSystem**s. Each action that occurs in the **GameWorld** is represented by a **TFEvent**. Each **TFEvent** has a **TFEntity**'s id associated with it which represents the **TFEntity** that the event is occurring on. These events, once created, are placed into the **EventManager** where they are added into an **EventQueue** which executes them in FIFO (First In First Out) order. Once called to execute, these **TFEvent**s will call the appropriate **TFSystem**(s), via the **EventTarget** protocol, which contain methods for the **TFEvent** to execute the action that they represent.

Below is a run through of the generation and resolution of a **DamageEvent**.

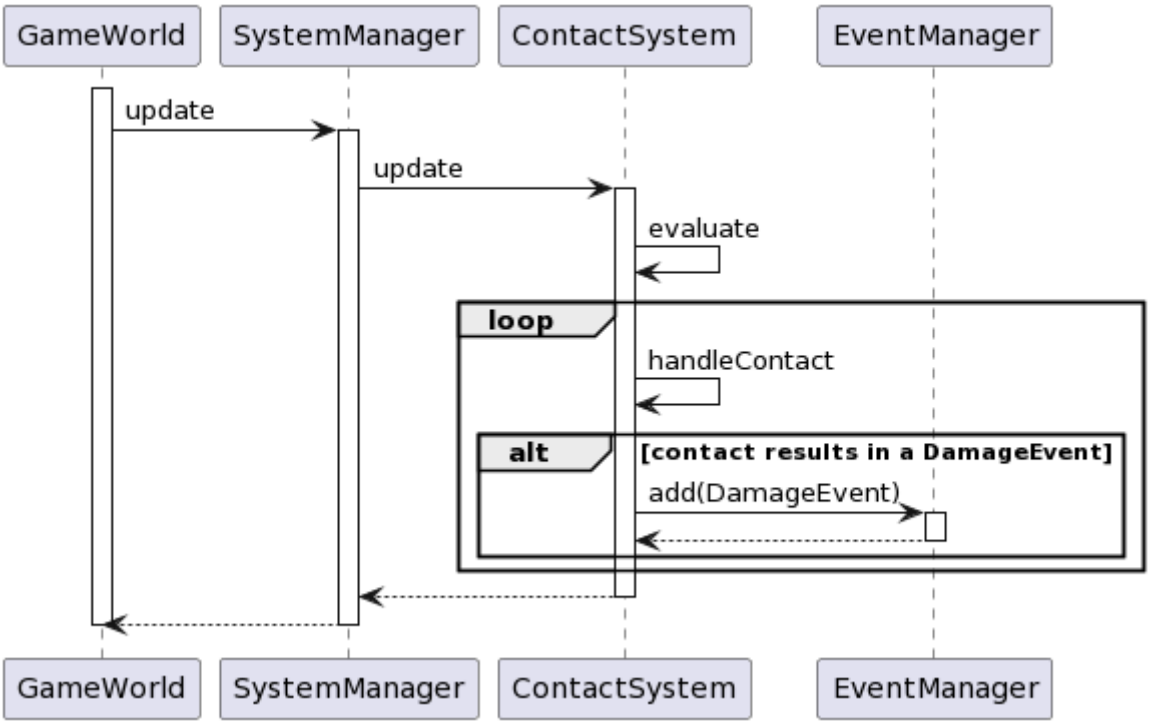


Figure 3: Sequence Diagram of Generation of DamageEvent

At every frame update called by the **GameScene**, the **GameWorld** calls for updates by all the **TFSystem**s that are attached to the **GameWorld.**

The **ContactSystem** iterates through all **TFEntities** that contain a **ContactComponent** and checks if there are any entities in contact. Using this, the **ContactSystem** invalidates any contacts for entities that are no longer in contact and evaluates contacts that are still occuring. This may result in the production of **TFEvent**s which is then added to the event queue in the **EventManager** to await execution.

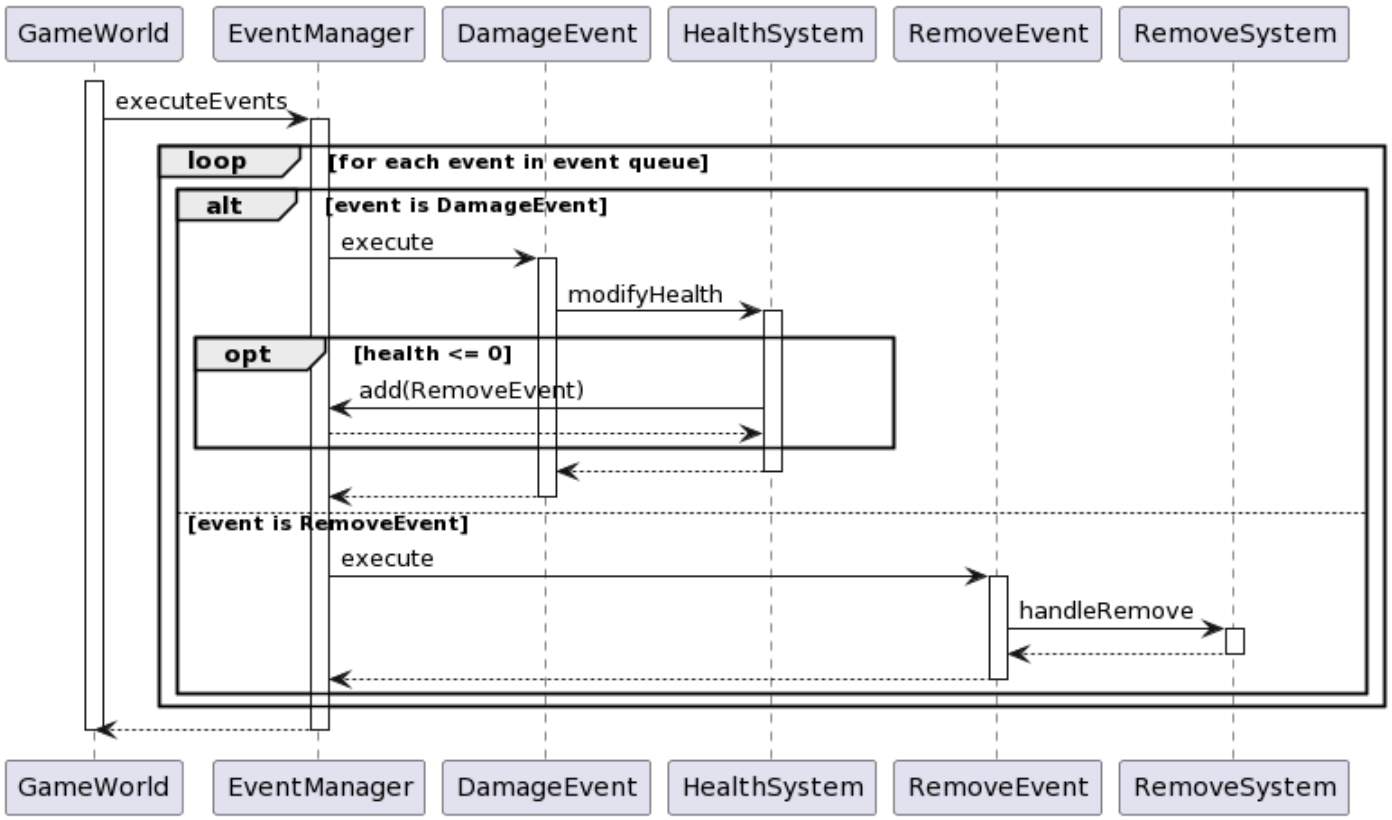


Figure 4: Sequence Diagram of Resolution of DamageEvent.

After all **TFSystem**s have finished their updates, the EventManager then executes all the **TFEvent**s that have accumulated in the event queue. If the **TFEvent** is a **DamageEvent**, it calls the **HealthSystem** to update the health of the damaged **TFEntity** that contains the id specified in the **TFEvent**. Once the health is updated, the **HealthSystem** returns the resulting health of the **TFEntity**. If the resulting health is less than or equal to 0, a **RemoveEvent** for the damaged **TFEntity** is returned by the **DamageEvent** to **EventManager** which adds the **RemoveEvent** to the event queue to await execution.

If the **TFEvent** is a **RemoveEvent**, it calls the **RemoveSystem** to remove the **TFEntity** with the id indicated in the **TFEvent** from the **GameWorld**. This would prevent other **TFEntities** from being able to interact with this **TFEntity** as well as being removed from the **Renderer** and will no longer be rendered.

Rendering API

The **Rendering API** is responsible for bridging the connection between the **TFEntities** that exist in the **GameWorld** and the **GameScene** which displays the **GameWorld** to the users. The **Rendering API** currently consists of just the **Renderer** and **Renderable** which is a protocol which exposes a method for **Renderer** to retrieve the **TFEntities** that they need to render.

The **Renderer** caches the **TFEntities** that have already been rendered and only updates them as needed. This reduces the cost of updating the view as compared to removing all existing view elements and rerendering all of them. The criteria for the action that the **Renderer** executes for each **TFEntity** is listed in Figure 6 below.

TFEntity	**TFSpriteNode**	**Description**	**Action**
Exists	Not cached	TFEntity not rendered	Add and Cache
Does Not Exist	Cached	TFEntity removed from world	Remove and Uncache
Exists	Cached	TFEntity rendered	Update

Figure 5: Criteria for Renderer Action

Since different **TFEntities** can different vastly from one another due to possibly having a very different set of **TFComponent**s, it is hard for the **Renderer** to be able to accommodate all the different behaviors and display them properly. Thus, in order to accommodate this complexity, the **Renderer** uses the **RenderStage**s that have been added to it to convert the **TFEntities** that exist in the **GameWorld** into **TFNode**s which can be rendered.

These **RenderStage**s acting as a **Render Pipeline** were inspired by the **TFSystem**s that are the core part of the **ECS** model. Just like how the **TFSystems** define the behavior of a **TFEntity** in the **GameWorld** depending on its **TFComponent**s, the **RenderStage**s define how the **TFEntity** looks when it is rendered. Each **RenderStage** operates on a single **TFComponent** type and ignores a **TFEntity** if it does not contain the required **TFComponent**.

Below is the Activity Diagram showcasing the flow of events in the **Render Pipeline**.

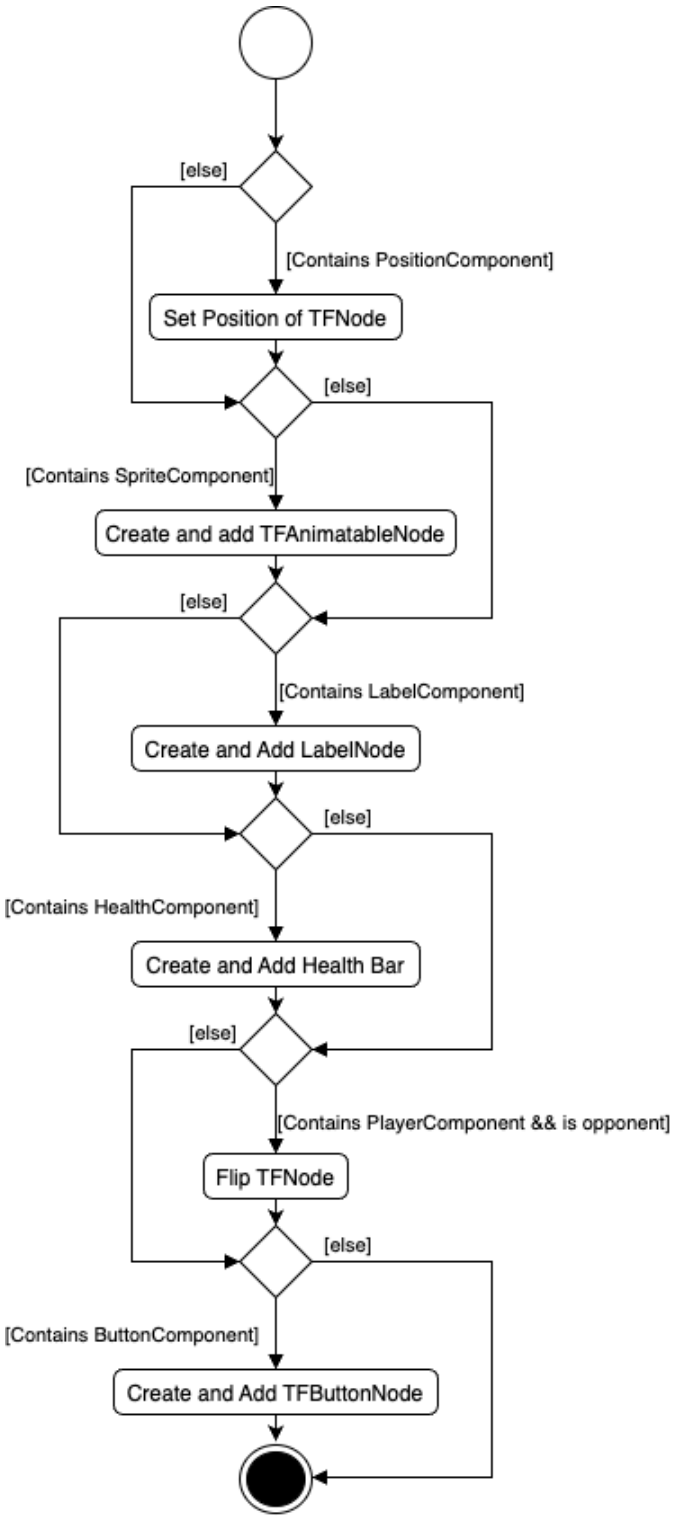


Figure 6: Activity Diagram of Render Pipeline

Storage API

The Storage API is responsible for providing the data persistence functionality.

The Storage Architecture of TowerForge consists of a few layers. From highest to lowest, these are:

- **StorageManager:** Interface that allows application wide access to data persistence
- **LocalDatabase:** A data structure that underlies storage manager, essentially a collection of Storages
- **Storage:** A collection of a specific type of items that can be written to file
- **Storable:** The lowest level of storage, represents a single unit of an item that can be written to file

This allows for a nuanced, sequential and hierarchical approach to data persistence, within a monolithic Storage Architecture, and thus, avoiding fragmented Storage across TowerForge. This also allows for StorageManager to transform into an adaptor if the need arises to replace FileManager with some other form of persistence, like CloudKit or Firebase or something else.

A singular LocalDatabase allows for simultaneously storing and isolating storable items of different types. A universal StorageManager allows module-wide access via its singleton instance (and internal-private to avoid other modules from accessing StorageManager). Thus, the component that provides the Data Access Functionality is separated from the component that provides the Local Storage Functionality, allowing each component to be replaced without

Hypothetical Example:

Local copy of the TowerForge application may contain information such as a list of Achievements and a list of user preferences.

- This would translate to a "AchievementStorage: Storage" class and "UserPrefStorage: Storage" class being stored within “LocalDatabase” inside the StorageManager and loaded upon every launch of the application.
- "Achievement: Storable" would be stored inside AchievementStorage and "UserPreferenece: Storable" would be stored within the UserPrefStorage.

This allows us to use a single storage architecture to store information of different types, whilst still having each data type compartmentalized.

Usage of Libraries

We utilized **SpriteKit**'s components such as **SKSpriteNode**, **SKTexture**, **SKLabelNode**, and **SKActions** to create adaptors for our game engine. For example, we developed **TFSpriteNode** and **TAnimatableNode** by building upon **SKSpriteNode** and **SKAction**, respectively. These adaptors provided additional functionality and customization options tailored to our game's specific requirements. With **TFSpriteNode**, we extended the capabilities of **SKSpriteNode** to suit our game's rendering needs, while **TAnimatableNode** allowed for more sophisticated animations and visual effects to render the game units.

Designs Problems

Network API

Since we wanted our game to be more accessible, we opted not to use an authentication system, allowing any player to be able to access our game with just a nickname. However, because of this decision it is hard to determine the identity of the player from anywhere in the app. Due to this, we have to constantly pass the player id all around the game. Moreover, it is also hard to ensure the one publishing the **TFRemoteEvent** to be one of the players in the **GameRoom**.

Moreover, currently there is no system in place for synchronized start of the game. Hence, any player that is able to join earlier will have a slight advantage due to having more time to accumulate the currency needed to purchase units and towers. Hence, there is a need to implement a system in the **GameRoom** for players to indicate that they are ready and to start the session automatically when all players are ready.

Lastly, currently each player has their own version of the game and no data other than spawns are shared between players. Hence, there is a possibility of each player’s games being different from one other, making it hard to determine the correct series of events and resolve the winner. Hence, to combat this, there is a need to assign one of the players as the host and their version of the **GameWorld** will be considered as the single point of truth. However, in order to achieve this, there is a need for more **RemoteEvent**s to be implemented to share information about the current status of the Host’s **GameWorld**.

Storage API

We wanted to implement the most versatile storage architecture, even though initially we had only wanted to store Achievements, initially. Thus, we devised a multi-layer architecture consisting of the Storable, Storage, LocalDatabase and the StorageManager. In order for each layer to be as separate as possible, each layer had its own encoding and decoding methods.

However, a limitation of Swift is that protocols cannot be coded directly, only a concrete implementation of the protocol can be coded. Thus, we had to turn Storable and Storage, which make more sense architecturally to be protocols, into concrete classes in order to comply with this requirement.

5 – Runtime Structure

Relevant Classes

GameWorld

GameWorld is where the EntityManager, SystemManager, EventManager is initialized and set up. In the initialization of GameWorld, we set up the various systems present in the game along with the systems manager which is used to process systems, the event manager which is used to process events, and the entity manager which is used to add the initial entities in the game. For instance, we set up the teams and player information (points, life) on game initialization. We also add the various systems using the initialized system manager.

EntityManager

We use an EntityManager to manage the entities in the game. The EntityManager contains an entitiesMap which uniquely identifies a TFEntity with a UUID. The systems can then call on the entity manager to perform operations (retrieval, addition, deletion) on the components in the ECS module.

SystemManager

The SystemManager is used to manage the various systems present in the game. The SystemsManager is responsible for adding or deleting systems, which is done on game initialization. Then, its primary function during the game run is to update the various systems. The various systems are responsible for their own updates. For instance, the MovementSystem will handle the movement of the movable components in the different entities present in the game.

Implementation of CheckRepresentation

Overview

EntityManager and TFEntity both contain a dictionary where the key is the UUID of the element stored. EntityManager contains a dictionary called “entitiesMap” whose key is of the type UUID and the corresponding value is a TFEntity. Since each TFEntity contains a UUID as its unique identifier, this allows for fast retrieval of a given TFEntity, with O(1) average time complexity. It is possible to query a TFEntity without knowing any information about it. The same principle can be generalized to TFComponent as well.

Runtime Vulnerability

The universality of UUID and its inherent uniqueness (without requiring a central authority to validate its uniqueness) allows for multiple modules or systems that encapsulate TFEntity to exchange information about a specific TFEntity without directly exchanging copies of a TFEntity itself, by simply passing the UUID associated with that entity. However, this opens up the possibility of UUIDs being mutated and thereby introducing a vulnerability in the system. Even though the UUID of each TFEntity is a *let* constant, the keys can be mutated resulting in a situation where one the key doesn’t correspond to the id within the TFEntity that is the value corresponding to the key. The same principle can be generalized to TFComponent.

CheckRepresentation Function

To fix this vulnerability, a checkRepresentation function is introduced wherever there is a usage of a value whose id is the key of a dictionary data structure used elsewhere in the codebase.

```
74      private func checkRepresentation() -> Bool {
75          for (key, value) in components {
76              if key != components[key]?.id {
77                  return false
78              }
79          }
80          return true
81      }
82  }
```

Usage

This function can then be called to assert wherever that particular data structure is used. In our case, this corresponds to wherever TFEntity and TFComponent are contained, which are EntityManager and TFEntity respectively. Whenever this data structure is modified, the checkRepresentation function can be asserted. The private nature of the function ensures that the integrity of the data structure can be ascertained without ever having to expose that data structure to test modules.

TFEntity

```
33      func addComponent<T: TFComponent>(_ component: T) {
34          /// assert(checkRepresentation())
35          guard !hasComponent(ofType: type(of: component)) else {
36              return
37          }
38          component.didAddToEntity(self)
39          components[component.id] = (component)
40      }
41
42      func removeComponent<T: TFComponent>(ofType type: T.Type) {
43          /// assert(checkRepresentation())
44          guard let componentToBeRemoved = component(ofType: type.self) else {
45              return
46          }
47          componentToBeRemoved.willRemoveFromEntity()
48          components.removeValue(forKey: componentToBeRemoved.id)
49      }
```

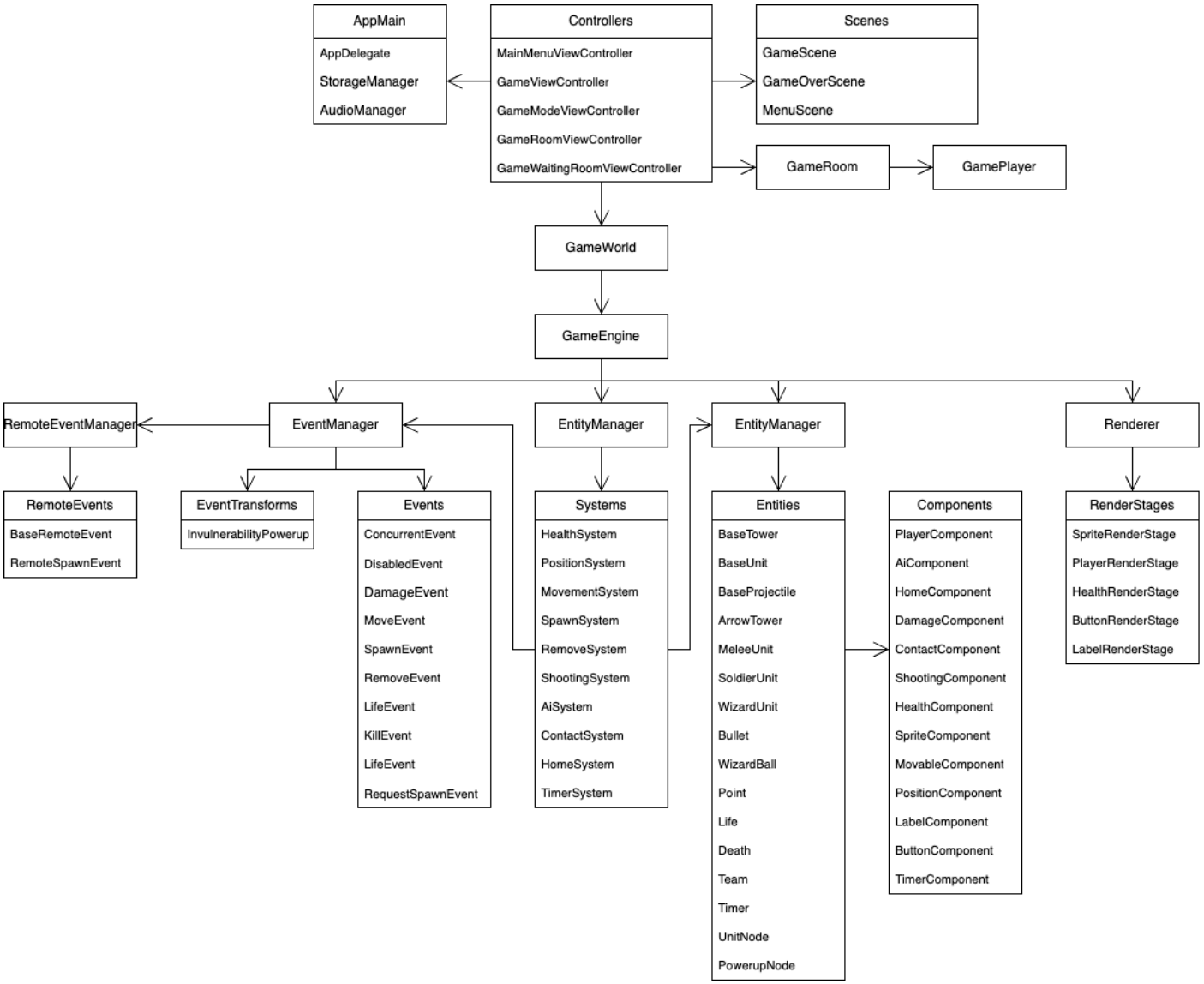
Entity Manager

```
25      func add(_ entity: TFEntity) {
26          entitiesMap[entity.id] = entity
27          /// assert(checkRepresentation())
28      }
29
30      func removeEntity(with id: UUID) {
31          entitiesMap.removeValue(forKey: id)
32          /// assert(checkRepresentation())
33      }
```

6 – Module Structure

Module Structure and Diagrams

Below is the module diagram for the game:



ECS Framework
TowerForge follows the ECS framework by separating the fundamental design into three parts : Entities, Components and Systems. More about this, including the design patterns used are explained in the Design overview section.

TFComponent represents a characteristic or trait that the game entity has. Each component can override the update(deltaTime:) method to make updates to the entities and components which work well with the **Game Scene**‘s update method.

TFSystem is where the component system lies and is responsible for handling updates. Each TFComponent will have an associated TFSystem.

Unit Generator is used to spawn the units into the game. In this class, we have methods to spawn different units.

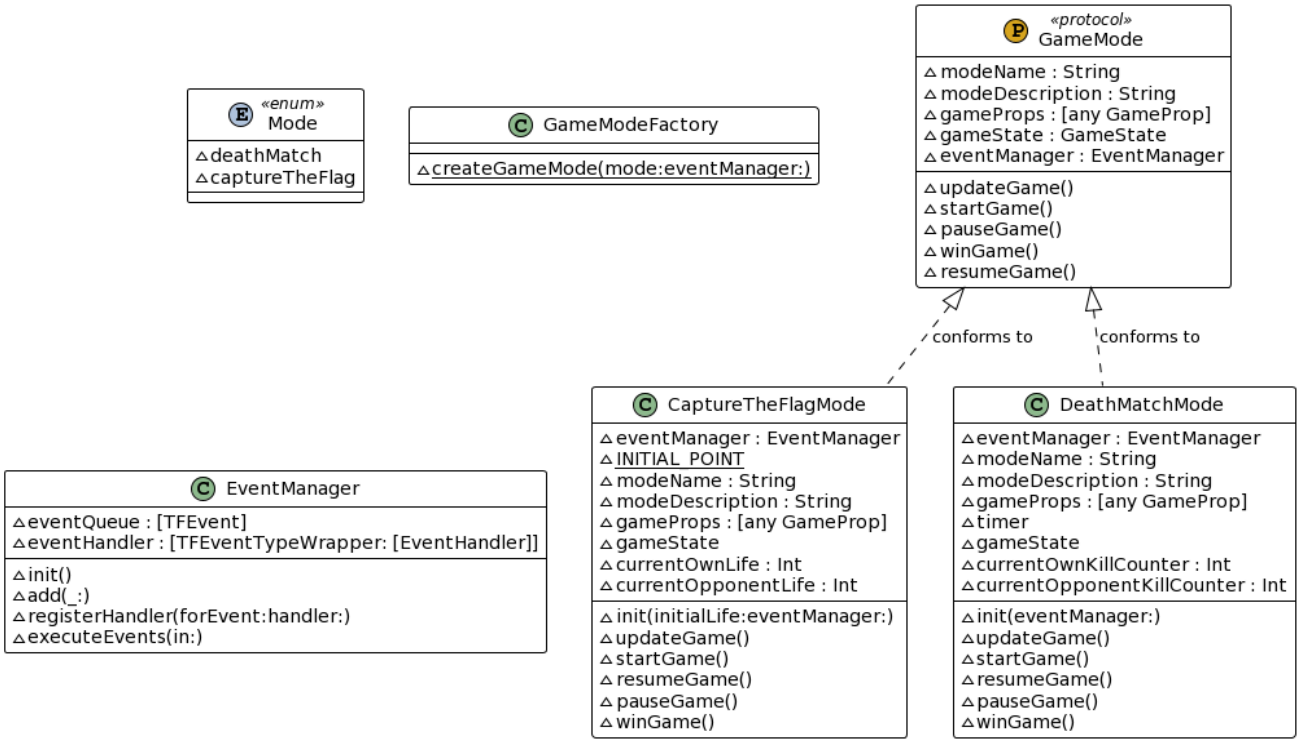
TFEvent represents an event that will be executed in the game. The game uses events to process changes in the game where each event corresponds to a single change in the game. For instance, the spawn event will spawn a unit in the game upon execution.

UnitSelectionNode is used to select the player's unit that will be deployed in the game. The player can choose from the units that are available in the game and this will be reflected accordingly on the screen.

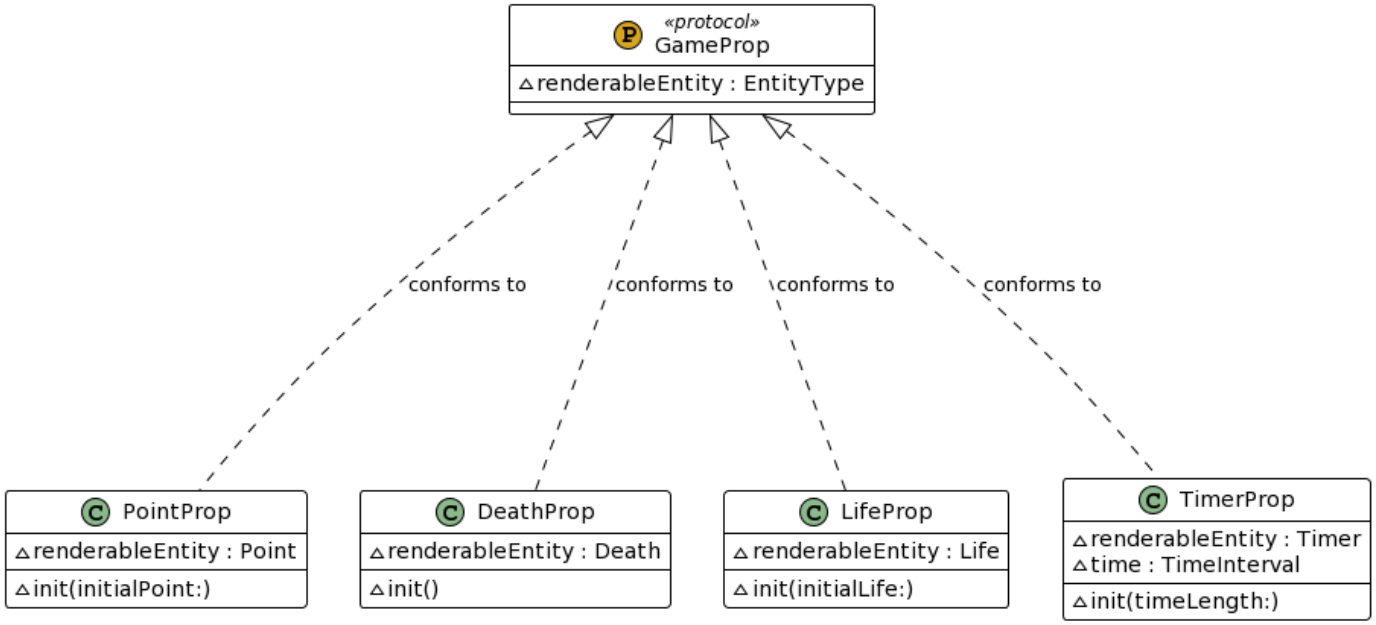
StorageManager is used as the adaptor for data persistence. The application access StorageManager via a single-ton instance.

AchievementManager handles all achievement-related activities such as loading and updating Achievements. **Achievements** conform to the **Storable** protocol which allow them to be encoded and decoded with StorageManager.

Game Modes



The **GameMode** protocol defines the specific game play and objective of the game. The game modes that conform to this protocol like the **DeathMatchMode** class will then be able to offer unique challenges. The **GameModeFactory** implements the Factory pattern that is responsible for creating instances of different game modes based on the type. This then helps to promote modularity and extensibility hence allowing us to easily add new game modes in the new future. Each game might have different HUDs and features and hence the **GameProp** protocol is added to encapsulate the behavior of in-game HUD entities that serve as a static metric to the players.



Hence, we can easily add new properties and customize the gameplay elements easily as needed. Furthermore, within the initialiser of each **GameMode**, event handlers are registered to the gameplay where certain callbacks are resulted if the specific event type happens in the game world.

```
eventManager.registerHandler(forEvent: KillEvent.self) { event in
            if let killEvent = event as? KillEvent {
                // Check if the event reduces life
                if killEvent.player == .ownPlayer {
                    self.currentOpponentKillCounter += 1
                } else if killEvent.player == .oppositePlayer {
                    self.currentOwnKillCounter += 1
                }
            }
        }
```

For instance, in the **DeathMatchMode**, when a **KillEvent** is triggered, the event handler updates the kill counter for each player. Here, the **EventManager** bridges the gap between event-driven interactions and logic within each game mode that might influence the progression and outcome of the game.

Game Room

The **GameRoom** class serves as a backbone to manage the players within the rooms. Before creating a room, the initializer will check if there is such an existing room in the database already in order to prevent duplicate rooms. If it is available, then data will be posted to the Firebase and a listener will be set up to listen for changes in the database. These changes are established through the makeRoomChangeListener method. The **GameRoomDelegate** protocol enables the other objects to receive notifications when the room changes so that the delegates can react to events and update the UI when the player joins or leaves the room.

Game Network

The game network serves as the form of communication between players playing in the same **GameRoom**. They do so via the use of **TFRemoteEvent**s. Each player will have their own **EventManager** that will contain a **RemoteEventManager** as well. This **RemoteEventManager** will contain a

TFRemoteEventPublisher and a **TFRemoteEventSubscriber**, allowing each player to upload and receive **TFRemoteEvents** from the other player. For example, if a player attempts to spawn an entity, it will create a **RemoteSpawnEvent** and publish it to the network server via the **TFRemoteEventPublisher**. This will then be propagated to all players that are subscribed to the same **GameRoom**, allowing them to receive the **SpawnEvent**, spawning the appropriate entity on their local copy of the game.

Power Ups

To facilitate the addition of power-ups in the game, we added an event pipeline under **EventManager** that events have to undergo before execution. Each event in the event queue will undergo any **EventTransformation**s that are present in the list of event transformations under **EventManager**. Due to the clear separation of concerns between the Entity Component System and the Events API, all events in the game are handled by the **EventManager**. Hence, each power up will not have to gain access to the relevant systems and can instead be defined as **EventTransformation**s on the game events that it affects. In this way, the game behavior can be modified by power ups without having to interface with the entities themselves. Furthermore, each power up is singularly defined and conforms to the **EventTransformation** protocol, so we can utilize polymorphism to define power ups.

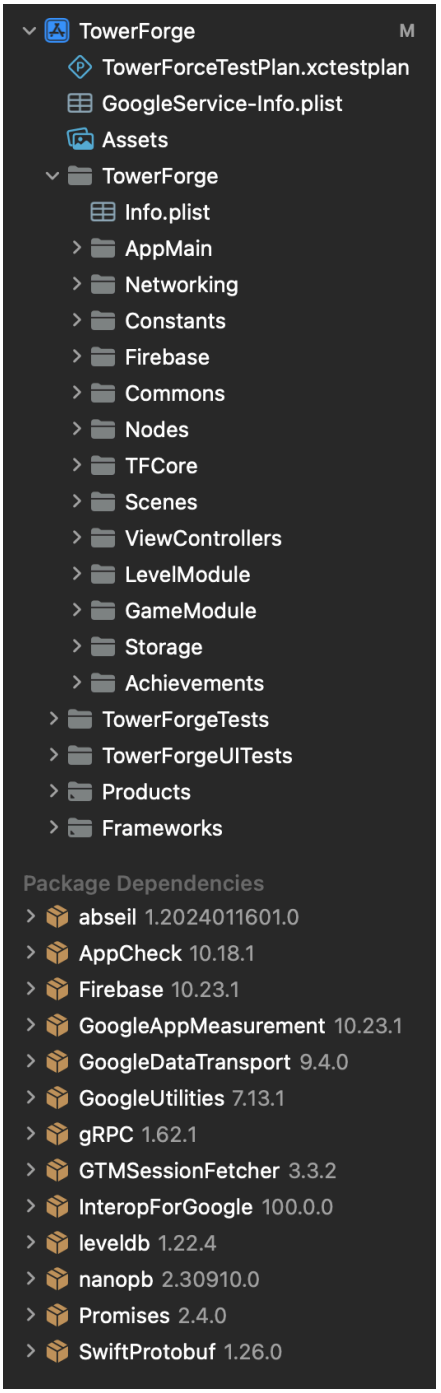
To activate power ups, we used the delegate pattern which will be called by the respective UI elements to activate the power ups. Each power up will define the duration for which it is active, and the delegate is also responsible for removing the power up after the duration has passed. We also utilized the factory pattern to initialize the power up delegates in the selection node.

Invulnerability Power Up

Each damage event in the game is handled by the execution of a **DamageEvent**. When a unit collides with another unit, a **DamageEvent** is created and appended to the event queue. When the invulnerability power up is active, all **DamageEvent**s on the player's units are disabled and transformed to a **DisabledEvent**. Hence, the player's units become invulnerable for a set period of time, defined in **InvulnerabilityPowerUp**.

File Structure

We've organized the project as shown on the right. The TowerForge root folder contains the majority of the project file. Below are some files and groups explained (including those not included in the above module diagram).



TestPlan

The TowerForgeTestPlan file is the custom test plan we use, which allows us to have more control over the tests executed. This is particularly important because we use continuous integration to build and test our project using GitHub Actions, and having UITests run for every PR is very resource intensive and time intensive. Thus, we can use this custom TestPlan to disable UITests in the cloud while being able to run them locally when needed.

Fonts

This group contains the custom fonts that we use in TowerForge

AppMain

This group contains files related to the Application and App lifecycle, such as the AppDelegate class and storyboard files.

Protocols

This group contains protocols used universally across TowerForge, such as **Animatable** and **Spawnable**.

TFCore

This group contains base elements needed for GUI rendering, including some SpriteKit components.

LevelModule

This group contains files and classes responsible for the majority of game logic, including entities, components, systems, all the respective managers, and level-specific features such as the grid and unit generators.

GameModule

This group contains the GameWorld class which is responsible for the majority of the Game Logic, and other classes needed for game functionality

TowerForgeTests

This group contains unit tests and integrations tests.

TowerForgeUITests

This group contains UI tests.

7 – Reflection

Lessons

ECS is a very powerful design pattern for software that requires very customisable behavior. With **ECS**, in order to add new behavior to an entity, all that needs to be done is to create a new component to hold the data representation relating to the behavior and add a new system to update the components to reflect the expected behavior. This makes **ECS** very suitable for games where there are many different entities that might share various behaviors but have some differing behaviors. As such, it is hard to represent the relationship between the differing entities with just regular **Polymorphism**.

Since we are building a game that belongs to the Tower Defense genre, we decided to make use of the benefits of both **Polymorphism** and **ECS**. Since our game's entities can be clearly defined into 3 types with distinct behavior, we implemented the base class for each of these types. They are **BaseUnit**, **BaseTower** and **BaseProjectile**. **BaseTower** has a health and a position, **BaseUnit** is similar to **BaseTower** but it can also move and **BaseProjectile** does not have health but can move. By defining these base classes, we need not repeatedly write the same code for classes that clearly subclass these base classes while still being able to customize each of their behavior to the nature of **ECS**.

Evaluation

Working with Xcode

One thing that we had greatly underestimated was the issues that can arise from working on an Xcode project. In most other development environments, modified files are tracked directly and singularly, merged conflicts can be handled on a file-by-file basis. In Xcode however, new files are indexed within the .pbxproj file, where each file and the entire file hierarchy is represented in text, alongside the file itself within the project. This presented a major problem when we had to restructure the project's file structure, and that caused merge conflicts that could not be resolved at all. The merge conflict had been caused by the .pbxproj file which was modified due to changes in the folder structure. The corrupted .pbxproj file meant that the conflicting changes had to be manually added again. To avoid this issue in the future, we refrain from changing the project's file structure frequently and when we needed to do so, we ensure that no one had any changes on any existing branches.

ECS & OOP

Initially based on the project proposal period, we had planned to develop the game using a traditional **OOP** design, where entities were represented as classes with predefined properties and behaviors. The decision to adopt the Entity-Component-System (**ECS**) architecture in TowerForge stemmed from our group's recognition of its numerous advantages over a purely Object-Oriented Programming (**OOP**) approach.

By combining **OOP** and **ECS**, TowerForge achieved a balance between object-oriented design and data-driven architecture. **OOP** provided a familiar and intuitive way to define game entities and their behaviors, while ECS offered greater flexibility and scalability by separating data from behavior. This hybrid approach enabled developers to leverage the strengths of both paradigms, resulting in a well-structured, modular, and efficient game architecture.

The adoption of **ECS** required some adjustments to our development workflow and mindset, but ultimately, it proved to be a valuable decision. It allowed us to better manage entity behavior and interactions, reduce complexity, and facilitate the addition of new features with minimal disruption to existing code.

Overall Reflection

Tell me how you think you're doing so far. Are you able to meet your scheduled deadlines? Have you slipped? What did you do (or what do you plan to do) about the slippage?

We believe that we are doing quite well thus far, being able to keep up with our internal deadlines and our forecasted schedule for the project. There were some minor slippages here and there particularly when multiple people work on a single component or dependency, this would naturally introduce some bottlenecks as certain components have to be finished first before progress on the dependent component can be made. One example was that the AIComponent required Systems to be implemented first, which had not yet been implemented by the time AIComponent was completed. However, this also allowed us to rethink our design to be more compatible with SWE principles - we had to ensure that the components we implemented depended as little on other parts of the codebase as possible. In doing so, not only were we able to make progress more effectively, we also learned to appreciate the importance and motivation behind some SWE principles by applying them first hand.

If you don't think you're doing well, what went wrong? What are the problems that your group is currently facing? How do you plan to deal with these problems?

As most of us are new to iOS development, we are also new to working with the Xcode integrated development environment. While we had worked with it individually during the individual project in the first half of the semester, this project is a collaborative effort and multiple people working on the same Xcode project presented its own issues (highlighted earlier). However, with proper communication, we were able to sort out all issues that we might have had, ensuring a seamless dev experience for all team members.

What have you learnt in the last two weeks? How are you going to apply that newfound knowledge to the rest of the project?

We had learned the importance of transparent communication and keeping to a schedule, ensuring that everyone is aware of what everyone else is working on and ensuring that we are all on the same page. This not only allows us to figure out any issues that might arise in the course of us working on TowerForge, but also ensures that everyone has a pleasant experience working together.

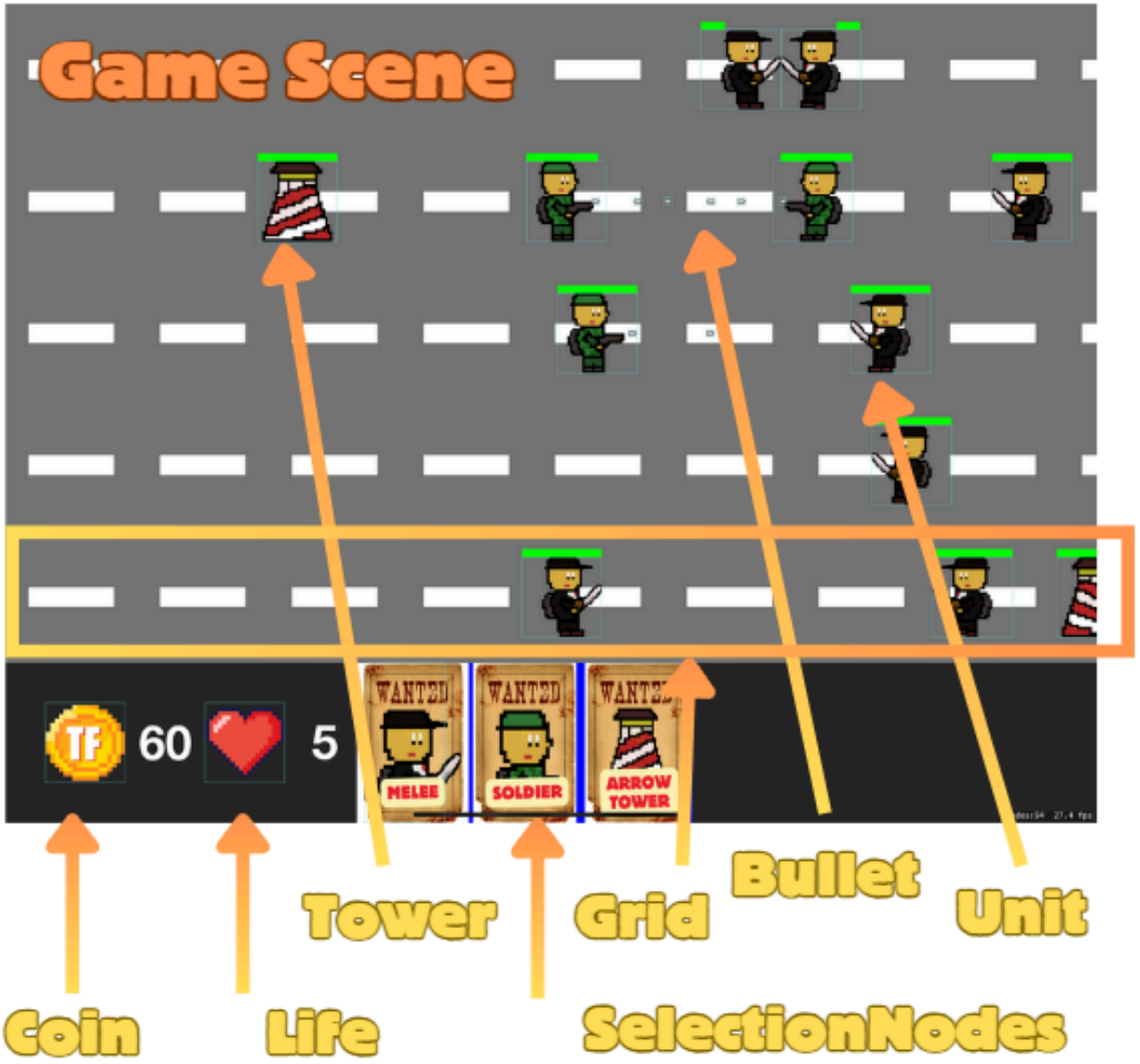
Known Bugs and Unresolved Issues

One current bug of the multiplayer system is that there is no current way to sync the start of the game for players. Players must start the game by themselves and if they do so later then the other players, they will face a disadvantage. Hence, there is a need for a ready-ing system and a system to start all players at the same time when all players are filled and ready.

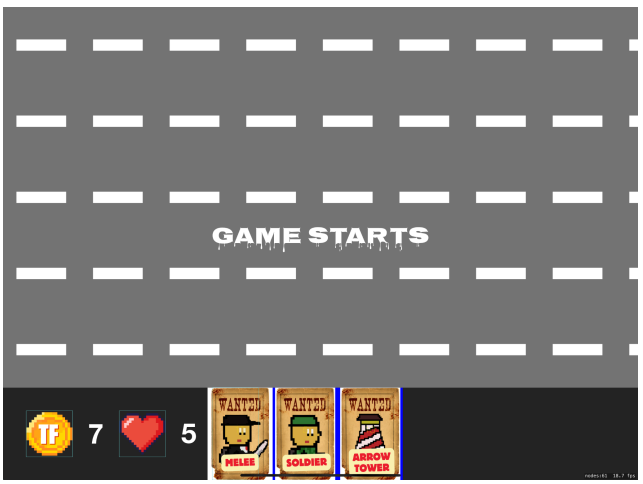
Another issue of the game currently is that there is currently only 1 supported **TFRemoteEvent** which is the **RemoteSpawnEvent**. While this is enough as a proof of concept and a MVP, as players can spawn entities onto each other's local copy of the game, there is still a need to synchronize other parts of the game. One such example is the removal of a killed entity and propagation of powerups to the other player. One player should be the host game and the state of the other player's local copy of the game should reflect that of the host.

8 – Appendix

A – GUI







B – Test Cases

Startup:

- Upon startup, the user should be redirected to the main screen where they can see a button “Play Game”
- Clicking on “Play Game” should redirect the user to the game screen

Game Screen:

- The user should see their available coins and life points on the bottom-left corner of the screen
- The user should be able to select between the different units (Melee, Soldier, Arrow Tower)

Spawning Units:

- With a unit selected, the user should be able to click on the screen to spawn the selected unit on a lane
- Upon spawning a unit, the available coins should decrease by the cost of the unit

Combat:

- All units should stop on contact with an enemy unit on the same lane
- Unit health points should decrease upon being attacked by an enemy unit
- When a unit’s health reaches zero, the unit should be removed from the game screen

Unit Behavior:

- Soldier units should begin shooting bullets when they are in range of an enemy unit on the same lane
- Melee units should attack on contact with an enemy unit
- Arrow Towers should block enemy units from advancing

Unit Tests

Unit tests have been written for **TFEntity**, **TFComponent**, **EntityManager**, **SystemManager** and **EventManager**.

GameMode

1. Initialization of the game mode
    a. Purpose: Ensures that the **GameMode** instance is initialized correctly with the expected initial properties and state
2. updateGame()
    a. Purpose: Verify that the game state is updated correctly based on the current conditions during the game play
        i. Scenario: Game timer runs out
            1. Expected: Game state should be set to a result based on the game conditions like (WIN, LOSE, DRAW)
        ii. Scenario: Game being paused
            1. Expected: GameMode properties that should be updated gets stopped

GameModeFactory

1. createGameMode()
    a. Purpose: Verify that the **createGameMode** function handles all possible cases defined in the **Mode** enum and returns the game mode instance that is supposedly the right subclass of the **GameMode**

GameRoom

1. Initializing the **GameRoom** class
    a. Purpose: Ensure that the **GameRoom** instance is initialized correctly and returns nil if the room is not available
        i. Scenario: Room is available
            1. Expected: Returns a GameRoom instance
        ii. Scenario: Room is not available
            1. Expected: Returns nil and completion is set to false because it has failed to initialize a gameRoom
2. joinRoom()
    a. Purpose: Verify the functionality of the **joinRoom** method when adding players to the room
        i. Scenario: Attempting to join a full room
            1. Expected: The player should not be added to the room and the completion handler should return false
        ii. Scenario: Attempting to join a non-full room
            1. Expected: The player should be successfully added to the room as either the playerOne or playerTwo.
3. leaveRoom()
    a. Purpose: Verify the functionality of the **leaveRoom** method in removing players from the room
        i. Scenario: Player leaves a room with two players
            1. Expected: the player should be removed from the room.
        ii. Scenario: Player fails to leave the room
            1. Expected: the completion handler should be returning false to indicate unsuccessful leaving of the room
4. isRoomFull()
    a. Purpose: Ensures that it correctly determines if the room is full
        i. Scenario: Check if a room with two players is considered full.
            1. Expected: The method should return true.

        ii. Scenario: Check if a room with one player is considered full.
            1. Expected: The method should return false.
5. makeRoomListener() + makeDeletionRoomListener()
    a. Purpose: Ensure that the room listener is called when the initializing is successfully done.

Integration Testing

Game Room Integration Tests

1. Room change notification for **GameRoomDelegate**
    a. Purpose: Ensure that the delegate receives notifications for room changes
        i. Scenario: Add a player in the database
            1. Expected: The GameRoom listens for changes and notifies using the GameRoomDelegate to send changes
2. Room Finding
    a. Purpose: Validate the integration between the findRoomWithName method and database retrieval.
    b. Steps:
        i. Create a room in the database with a specific name.
        ii. Use the findRoomWithName method to retrieve the room.
        iii. Verify that the retrieved room matches the expected room state.
3. Room Deletion
    a. Purpose: Validate the integration between the leaveRoom method and room deletion from the database.
    b. Steps:
        i. Create a room instance.
        ii. Join the room with multiple players.
        iii. Leave the room for each player until the room becomes empty.
        iv. Verify that the room is deleted from the database.
4. Joining and Leaving Room
    a. Purpose: Verify the integration between the joinRoom and leaveRoom methods along with the database operations.
    b. Steps:
        i. Create a room instance.
        ii. Join the room with multiple players.
        iii. Ensure that players are correctly added to the room and the database.
        iv. Leave the room for each player.

C – Project Schedule

Sprint	Week	Date	Task
-	Week 7	March 3	Admin Tasks - Set up Workflow, CI/CD, etc
1	Week 8	March 10	Models (Grid, Game Entities, Persistence) + Unit test
	Week 9	March 17	Game Loop, Event Queue, Game Engine, UI + Controllers
2	Week 10	March 24	Behaviour Customisation (Types, Paths, Enemy Prioritisation)
	Week 11	March 31	Multiplayer Network Adapter* (or other extensions)
3	Week 12	April 7	Multiplayer Network Adapter* (or other extensions)
	Week 13	April 14	Finalize project and Bug Fixing

Sprint Objectives

Sprint 1 (March 10 - March 23)

- We want to have the basic features and have a robust skeleton of the game which will be easily extendable.
- We want to have rigorous unit testing for all the components that we implement.
- We want to have code coverage analysis and continuous integration workflows set up for the rest of the project.

Sprint 2 (March 24 - April 6)

- We want to have basic customisation and possibly implement one of the extensions.
- Developing and integrating into a working game.
- We want to begin integration testing for the features that we have implemented and set up a workflow to continuously test new features

Sprint 3 (April 7 – April 17 STePS)

- We want to include the bells and whistles + other extensions that would be viable and effective to be added to the game and also polish up the game for final distribution/release.
- Finalizing the game
- Closer to the deadline we would want to implement a feature freeze and work on perfecting the features that we already have, sorting out any possible issues.

D – Individual roles and contribution

Week 8 Todos

Task	Assignee	Status
TF Adaptors	Vanessa	Done
Components + TFComponents	Vanessa	Done
Entities + EntityManager + TFEntity	Zheng Ze	Done
Storage	TBC	Skipped
Grid class	Keith	Done
Level class	Keith	Skipped
Systems + TFSystem	Rubesh	Done
CI/CD + SwiftLint + Github setup	Rubesh	Done

Week 9 Todos

Task	Assignee	Status
Game Loop	TBC	Using SpriteKit Implementation
Game Engine	Rubesh	Done (Integrated with GameWorld)
Event Queue + Event Manager + Events	Zheng Ze	Done
GameScene + Game Controller	Vanessa	Done
UI + Level Designer Controller	TBC	Skipped
Enemy Spawner	Vanessa	Done
Collision	Zheng Ze	Done (Using SpriteKit's collision)
UI + Nodes	Vanessa	Done
Renderer	Zheng Ze	Done
GameWorld	Zheng Ze	Done

Week 10 + 11 Todos

Task	Assignee	Status
Paths/Types (Low Priority bc need LevelDesigner)	TBC	Skipped
Enemy Prioritization (Low Priority)	TBC	Skipped
Missions / Achievements	Rubesh	In progress
Game Modes	Vanessa	Done
Storage (Base Implementation with FileManager)	Rubesh	Done
Power Ups + Spells	Rubesh + Keith	Done – to add more power ups
Render Pipeline	Zheng Ze	Done
Event Pipeline	Keith	Done
Contact System	Zheng Ze	Done
Touch Up UI	Rubesh	In progress
Camera Panning	Zheng Ze	Done
Movement with MoveEvent (Low Priority If even doing)	TBC	Skipped
Player base and life reduction	Vanessa	Done
Remove out of bounds entities	Vanessa	Done
Spawn Location Restrictions	Zheng Ze	Done
Firebase Adapters	Vanessa	Done
RoomNetwork	Vanessa	Done
GameNetwork	Zheng Ze	Done
Game States + UI	Keith, Vanessa	Done
Sound Manager + Sound Handling	Rubesh	Done

Week 12 Todos

Task	Assignee	Status
Ready System + Synchronised Start	Vanessa	
More RemoteEvents to reflect status of host game	Zheng Ze	
Leaderboard + Firebase Authentication	Vanessa	
Achievements	Rubesh	
Statistics/Ranking	Rubesh	
Character Screen (i.e. Achievements List & Rank Showcase)	Rubesh	
More Power Ups	Keith	
Migrate Movement to MoveEvent	Keith	
Fix sprite layering	Zheng Ze	
Update UI to new design (+Fix UI resolution issue)	Rubesh	

References

https://digitalcommons.calpoly.edu/cgi/viewcontent.cgi?article=1138&context=cpesp